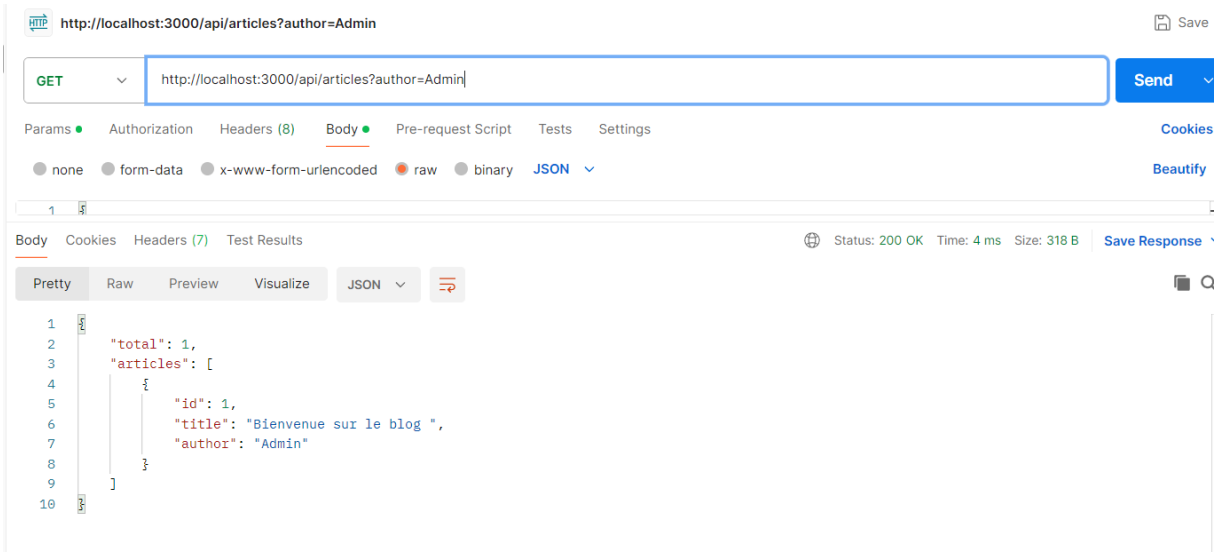
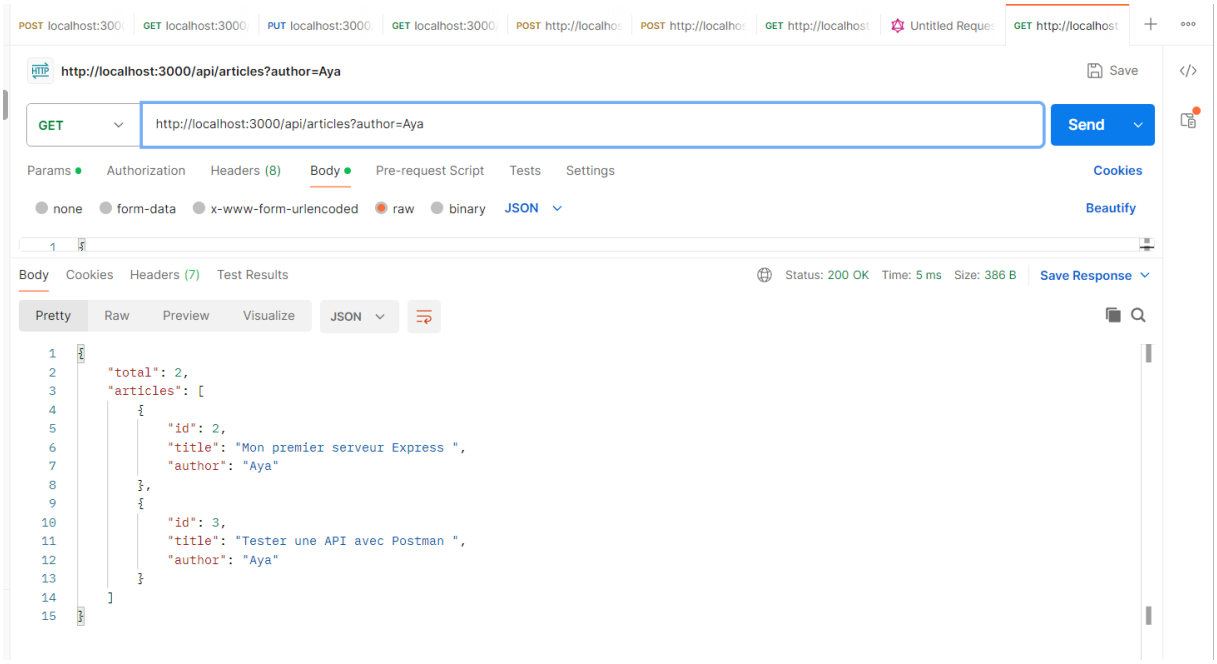
Khouloud Lassoued 5DS G2

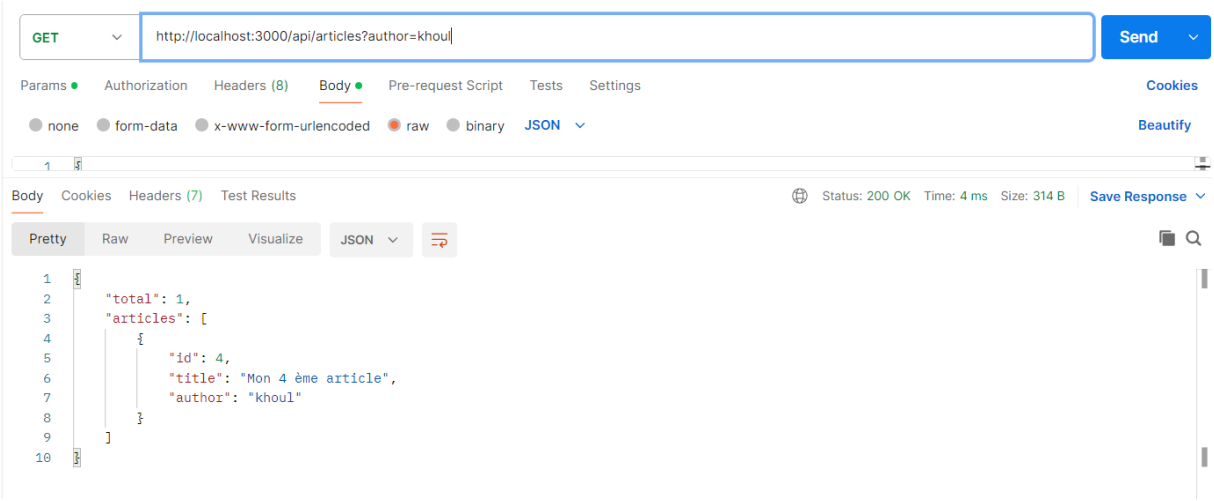
MERN Stack – Séance 1

Parie 2 Du TP1 :

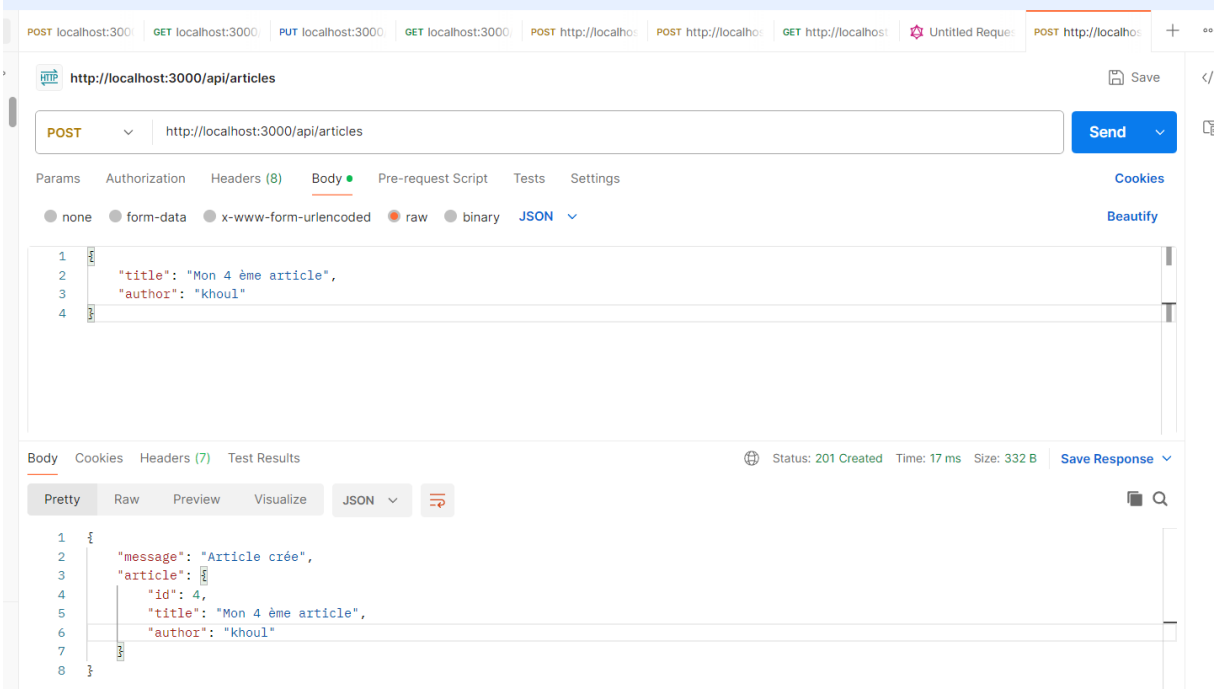
Étape 5 : filtrer par auteur

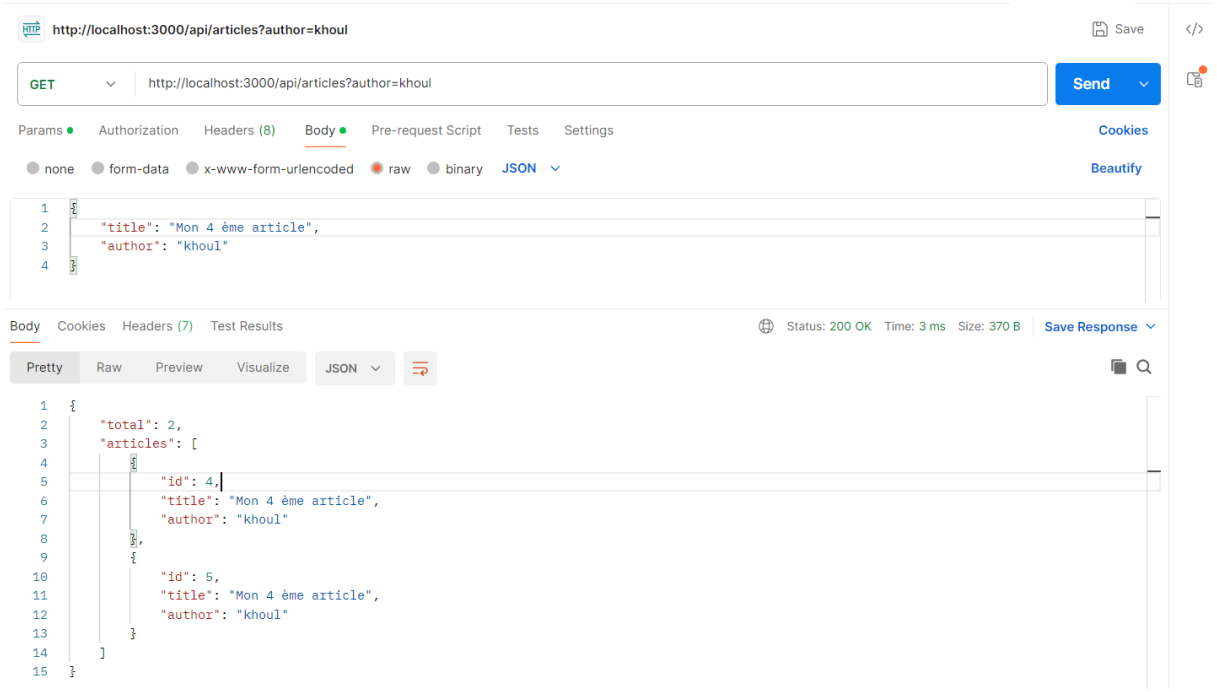






Étape 7 : tester avec Postman





Exercice 2 : les notions JavaScript de la séance

Code

```javascript
//----------------------------EX2 : destructuration d'objet et de tableau ----

const produits = [
    {nom: 'Clavier', prix:45 },
    {nom: 'Ecran', prix: 320},
    {nom:'Souris', prix:25}
]

// Pratie1 =>  Desctructuration : extraire les propriétés nom et prix du premier élément
const {nom, prix} = produits[0];
console.log(nom, prix);

//Partie 2 => Find
const produit_Souris = produits.find(p => p.nom === 'Souris');
console.log(produit_Souris.prix);

//Partie3 => Filter
const prixInf = produits.filter(p=> p.prix < 100);
console.log(prixInf);

//Partie 4 : Fonction fléchée
const avecRemise = prix => prix*0.90;
console.log(avecRemise(10));
```

Terminal :



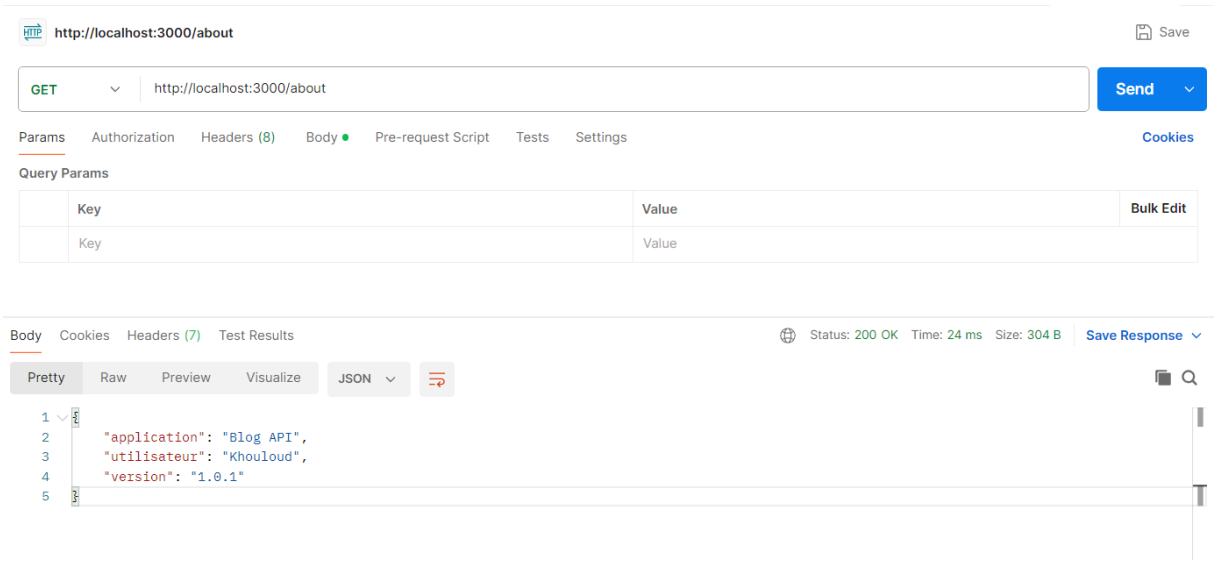
Exercice 1 : compléter l'API (dans server.js, à tester avec Postman)

1. ★ GET /about renvoie un objet avec le nom de l'application, votre nom et la version.

```
//------------------EX1: compléter l'API (dans server.js, à tester avec Postman)-------------
app.get ('/about',(req,res) => {
    res.json ({

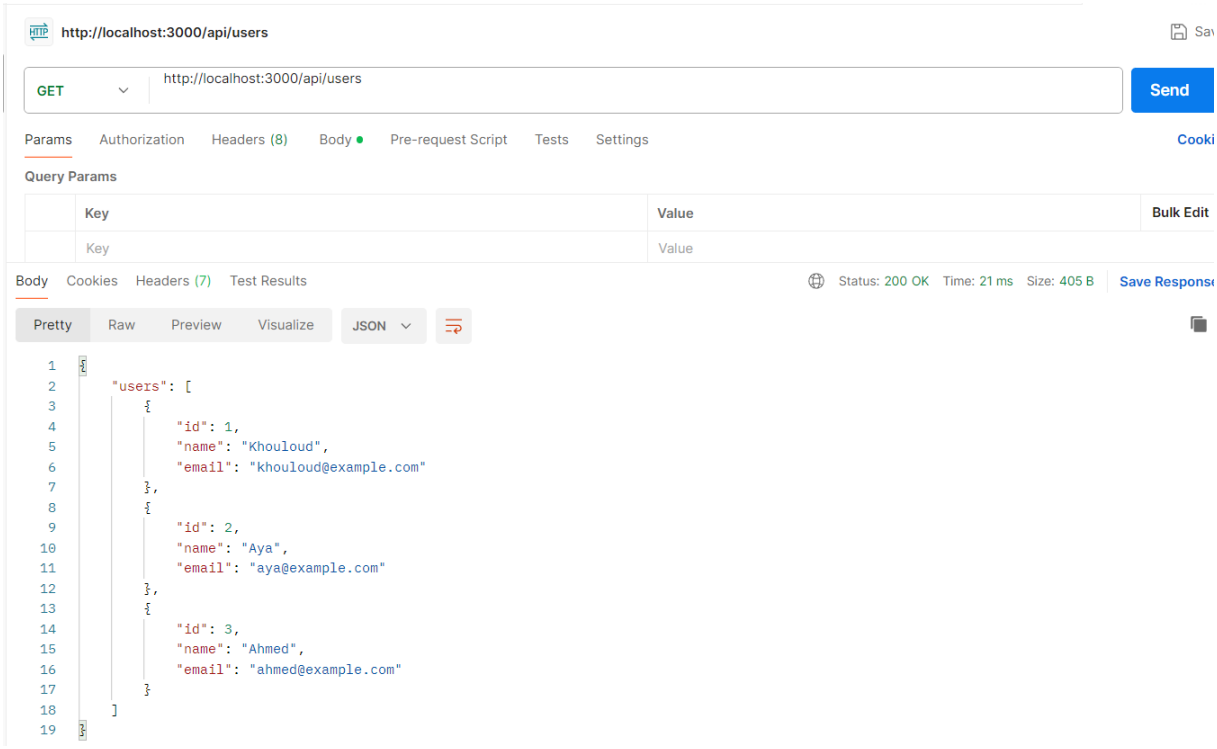
        application: "Blog API",
        utilisateur: "Khouloud",
        version:"1.0.1"
    });

});
```



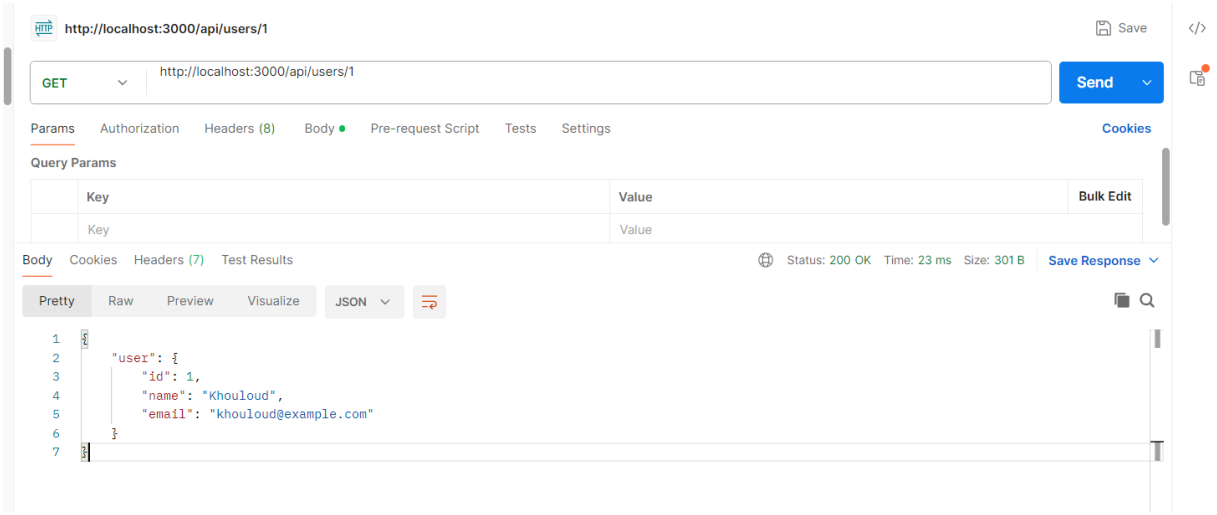
2. ★ GET /api/users renvoie un tableau de trois utilisateurs (id, name, email) rangé dans une constante users.

```
96
97  //-------------------------★★ GET /api/users-----------------------------
98  app.get('/api/users',(req,res) => {
99      const {name} = req.query;
100     let result = users;
101     if (name) {
102         result = users.filter (u => u.name === name);
103     }
104
105     res.json({users: result});
106
107 } )
108
```

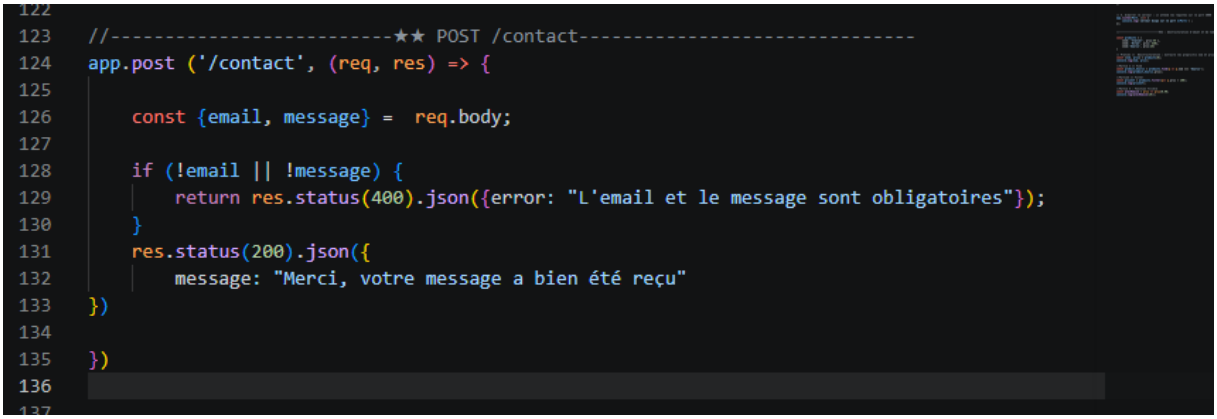


3. ★★ GET /api/users/:id renvoie l'utilisateur demandé, ou 404 s'il n'existe pas (inspirez-vous de l'étape 4).

```
110 //-------------------------★★ GET /api/users/:id-----------------------------
111 app.get ('/api/users/:id', (req,res) =>{
112
113     const id =Number (req.params.id);
114
115     const user = users.find (u=> u.id ===id);
116     if (!user) {
117         res.status(404).json({error: `Utilisateur ${id} introuvable`});
118     }
119
120     res.json({user: user});
121
122 })
```



4. ★★ POST /contact reçoit { "email": "...", "message": "..." }. Si un champ manque, répondez 400 ; sinon répondez 200 avec { "message": "Merci, votre message a bien été reçu" }.

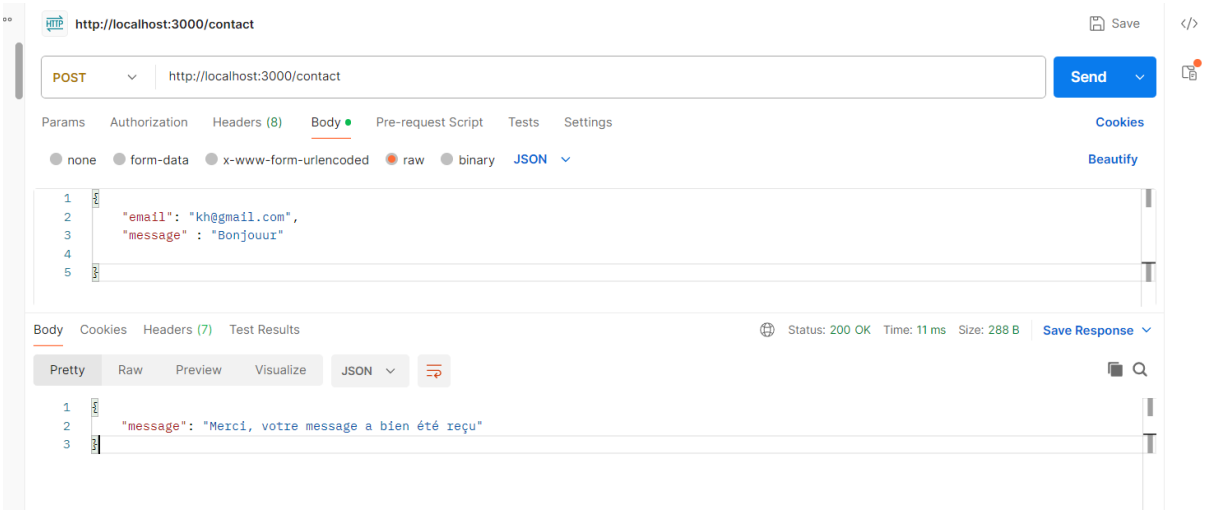


```
//-------------------------★★ POST /contact-----------------------------
app.post ('/contact', (req, res) => {

    const {email, message} =  req.body;

    if (!email || !message) {
        return res.status(400).json({error: "L'email et le message sont obligatoires"});
    }
    res.status(200).json({
        message: "Merci, votre message a bien été reçu"
})

})
```



5. ★★★ Bonus : GET

```
137
138  //-------------------------★★★★ Bonus : GET /api/users?name=Aya -------------------------
139
140  app.get('/api/users', (req, res) => {
141
142      const { name } = req.query;
143
144      let resultat = users;
145
146      if (name) {
147          resultat = users.filter(u => u.name === name);
148      }
149
150      res.json(resultat);
151  });
152
```

